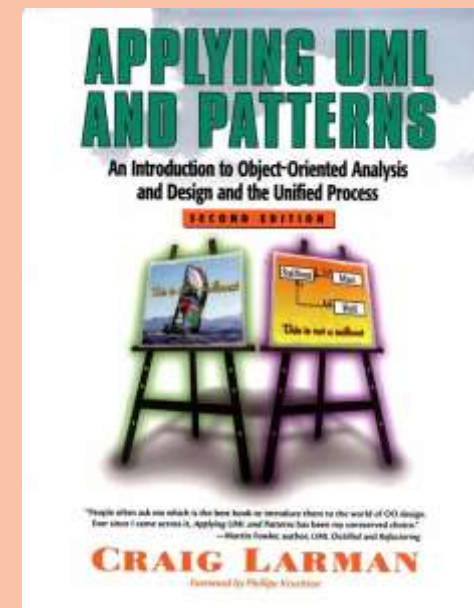


## *Writing Use Cases: Requirements in Context*

[www.craiglarman.com](http://www.craiglarman.com)

- Use cases are one way to capture (especially) functional requirements.
  - They are stories of using a system.
- You will be able to:
  - Identify different use case levels
  - Write use cases in the popular Cockburn format
  - Contrast essential and concrete use cases
  - Apply guidelines
  - Read and write use case diagrams

- Alistair Cockburn's approach is popular
- *Applying...* follows this style



Use case chapters were written with Cockburn

- Informally, a *use case* is a story of using a system to fulfill a goal.
  - *Rent Videos*
- Used by *primary actors*
  - *E.g., Clerk*
  - External agents
  - something with behavior
- Use *supporting actors*.
  - *CreditAuthorizationSystem*

- Here's one in a *brief format*:
  - **Rent Videos.** A Customer arrives with videos to rent. The Clerk enters their ID, and each video ID. The System outputs information on each. The Clerk requests the rental report. The System outputs it, which is given to the Customer with their videos.

- Informally, a *scenario* is a specific sequence of actions and interactions in a use case.
  - One path through the use case.
  - E.g., The scenario of renting videos and first having to pay overdue fines.
- More formally, a *use case* is a collection of success and failure scenarios describing a primary actor using a system to support a goal.



- There is nothing object-oriented about use cases.
- So, why bother in an OOA/D workshop?
  - We need some kind of requirements input for the design steps.
  - They are common/popular.
  - There is a UML-related topic.
    - Use case diagrams

- A common challenge is identifying use cases at a useful goal level.
- For example, how do we know which of these is at a useful level?
  - Negotiate a Supplier Contract
  - Rent Videos
  - Log In
  - Start Up





- One answer is that they are all use cases.
- Not helpful...
- We can end up with too many fine-grain use cases
  - management and complexity problems.
- Or “fat” use cases which span an entire organization.



- Cockburn supports the Elementary Business Process (EBP) guideline.
- Focus on use cases at the level of EBPs.
  - *“A task performed by one person in one place at one time, in response to a business event, which adds measurable business value and leaves the data in a consistent state.”*



- Naively, can you apply the “boss test” for an EBP?
- Boss: “What do you do all day?”
- Me: “I logged in!”
- Is Boss happy?





- An EBP-level use case *usually* is composed of several steps, not just one or two.
- It isn't a single step.

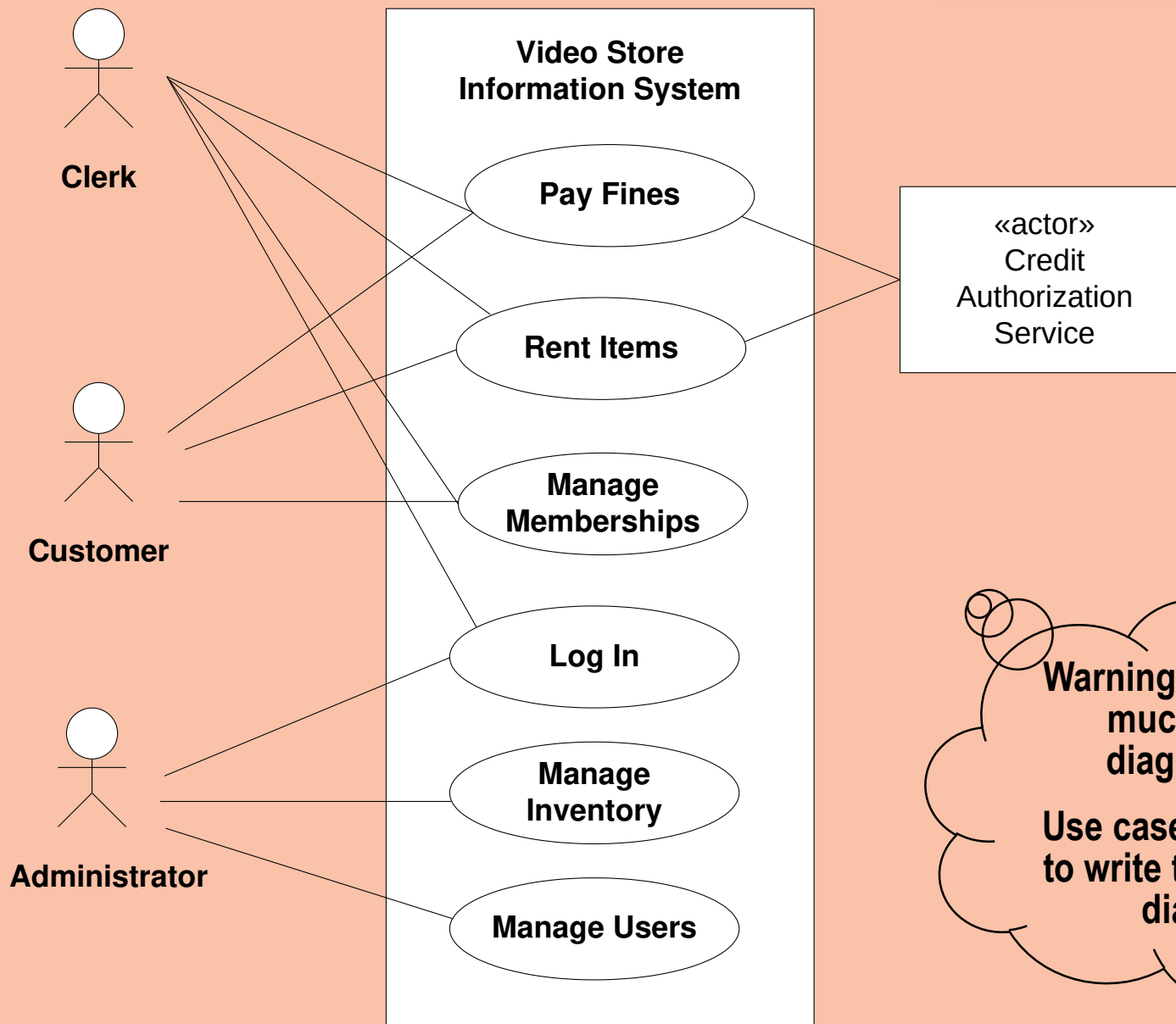


- Applying the EBP and size guidelines:
  - Negotiate a Supplier Contract
  - Rent Videos
  - Log In
  - Start Up
- The others *can* also be modeled as use cases.
  - But, prefer a focus on the EBP level.





- The UML has use case diagrams.
- Use cases are *text*, not diagrams.
  - Use case analysis is a *writing* effort, not drawing.
- But a *short* time drawing a use case diagram provides a context for:
  - identifying use cases by name
  - creating a “context diagram”



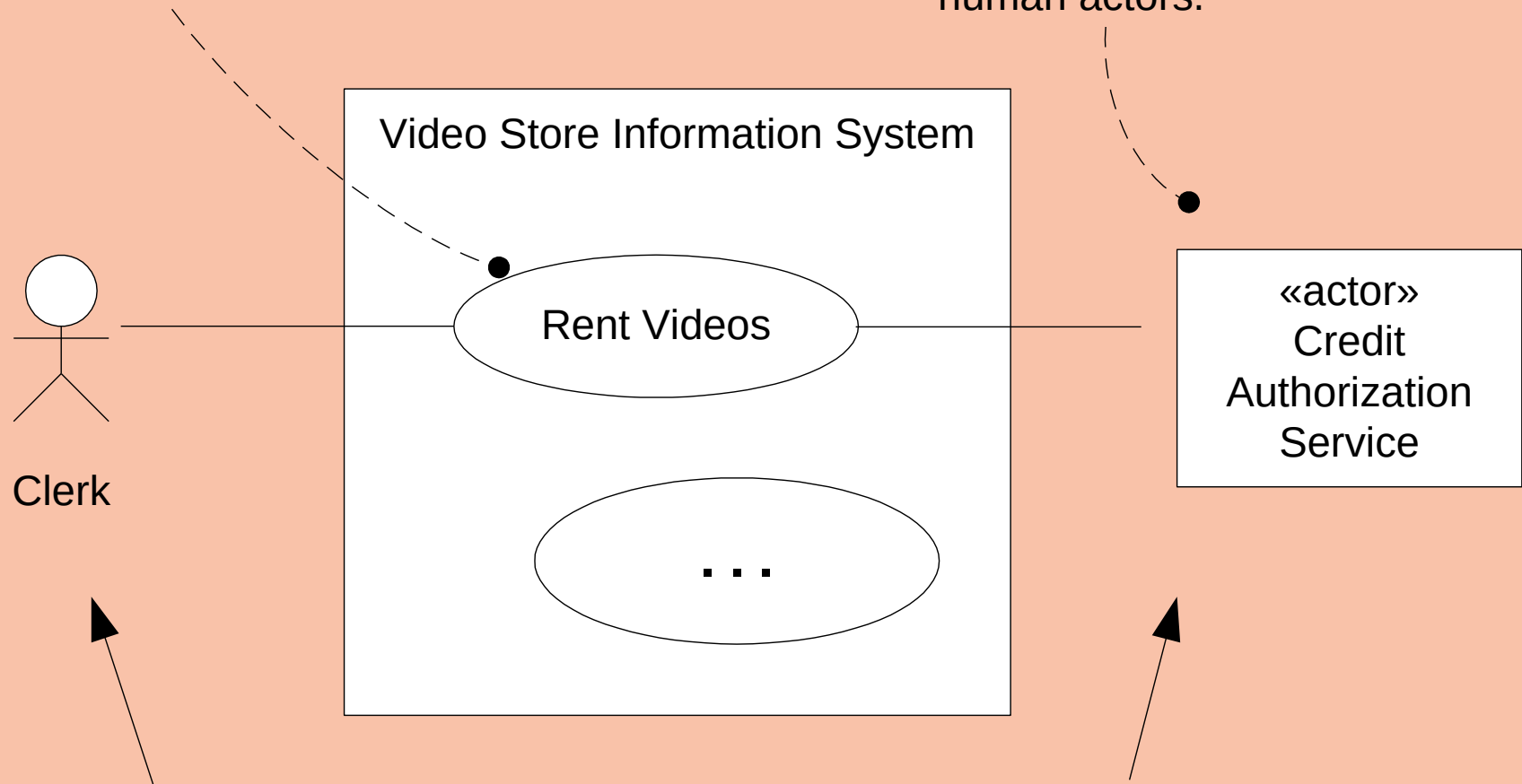
**Warning: Don't spend much time on diagramming.**

**Use case work means to write text, not draw diagrams**

# GUIDELINES: Use Case Diagrams

Prefer use cases at the EBP level.

Show computer system actors with an alternate notation to human actors.



primary actors on  
the left

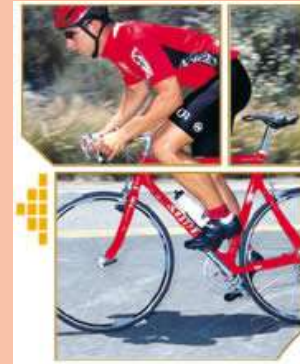
supporting actors  
on the right

- It is common to group CRUD operations into one use case.
  - Manage Users
- Name starts with a verb.
  - *Manage* Users
- All systems have a *Start Up* and *Shut Down* use case (perhaps trivial and low level).
  - But sometimes, important.
    - an avionics system



- Prefer EBP-level use cases.
  - AKA *user-level goal* use cases.
  
- Common quality assurance checks. Are these present:
  - Use Cases:
    - Some variant of *Configure System*
    - Sometimes, *Start Up* and *Shut Down*
  
  - Actors:
    - System Administrator

- In small teams at the white board, draw a use case diagram for the case study.
- Review and apply the guidelines.



- Rich notation for detailed analysis.
- Shows branching scenarios.
- Can include non-functional requirements related to the functional.

*Use Case UC1: Rent Video*

*Level: User-level goal (EBP level)*

*Primary Actor: Clerk*

*Preconditions:*

- Clerk is identified and authenticated.

*Stakeholders and their Interests:*

*Clerk: Wants accurate, fast entry.*

*Customer: Wants videos, and fast service with minimal effort.*

*Accountant: Wants to accurately record transactions.*

*Marketing: Wants to track customer habits.*

. . .



Main Success Scenario (or Basic Flow or “Happy Path”):

1. Customer arrives at a checkout with videos or games to rent.
2. Clerk enters Customer ID.
3. Clerk enters rental identifier.
4. System records rental line item and presents item description.  
(Clerk repeats steps 3-4 until indicates done.)
5. System displays total.
6. Customer pays. System handles payment.
7. Clerk requests rental report.
8. System outputs it. Clerk gives it to Customer.
9. Customer leaves with rentals and report.

Extensions (or Alternatives):

a\*. At any time, System fails:

1. Clerk restarts System
2. logs in
3. requests recovery from prior state

1a. New Customer.

1. Perform use case Manage Membership.

2a. Customer ID not found.

1. Cashier verifies ID. If entry error, reenter, else Manage Membership.

2b. Customer has unpaid fines (usually for late returns).

1. Pay Fines.

Special Requirements:

- n Language internationalization on the display messages and rental report.
- n Large text on display. Visible from 1 m.

Technology and Data Variations:

- n ID entries by bar code scanner or keyboard.

Frequency:

- n Near continuous

Open Issues:

- n Should we support a magnetic stripe cards for customer ID, and allow customer to directly use card reader?

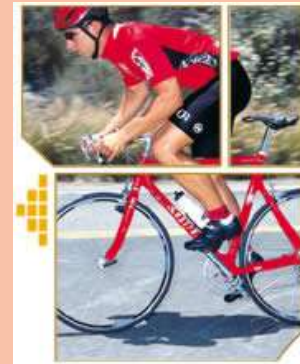
- “Keep the UI out”
- *Essential* use cases defer the details of the UI, and focus on the *intentions* of the actors.
- *Essential*: Clerk enters Customer ID.
- *Concrete/worse*: Clerk takes Customer ID card and reads the bar code with laser scanner.

- Start sentence 1 with “<Actor> does <event>”
  - Customer arrives with videos to rent.
  
- First write in the essential form, and “Keep the UI out.”
  
- Capitalize “actor” names.
  1. ...
  2. *Clerk* enters...
  3. *System* outputs...



- Terse is good. People don't like reading requirements ;). Avoid noisy words.
- More verbose
  1. ...
  2. The Clerk enters...
  3. The System outputs...
- Less
  1. ...
  2. *Clerk* enters...
  3. *System* outputs...

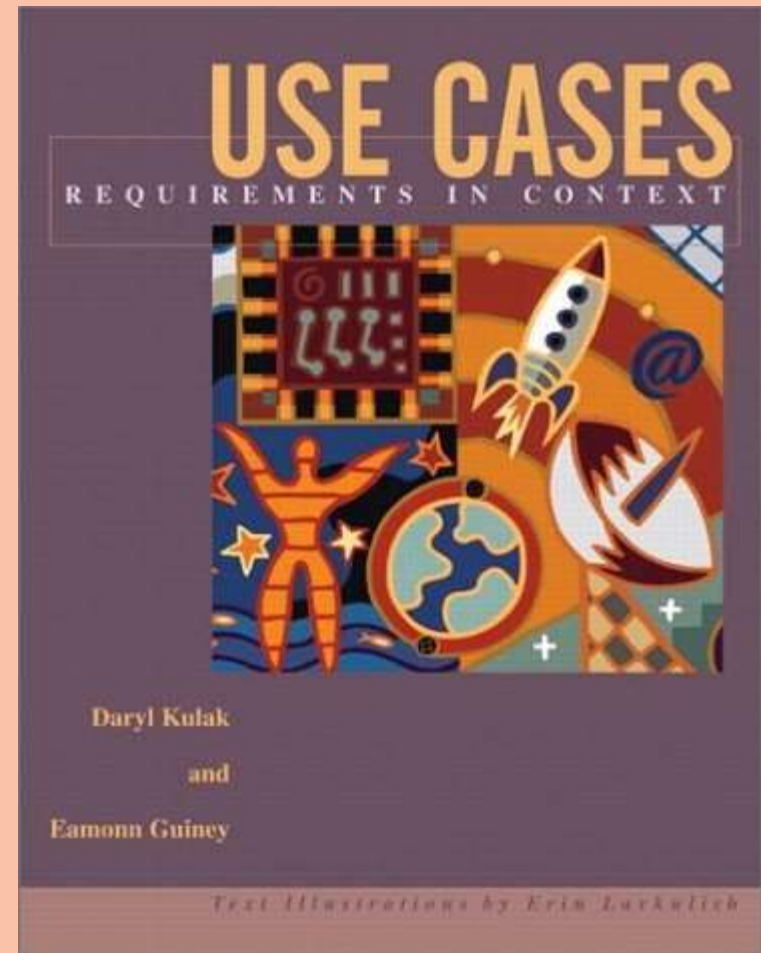
- In small teams at the white board, write a fully dressed use case for *Process Sale*.
- Review and apply the guidelines.



- Use cases are stories.
- A simple and familiar model that many people, especially non-technical, can easily relate to.



- The subtitle makes an important point:
- Use cases bring together related requirements.
- More cohesion and context for related requirements.



- Sometime after the essential form of the use case has been written, one may *optionally* write it in a concrete form.
- 1. Customer arrives at a checkout with videos or games to rent.
- 2. Clerk *scans* Customer ID...

## Extensions

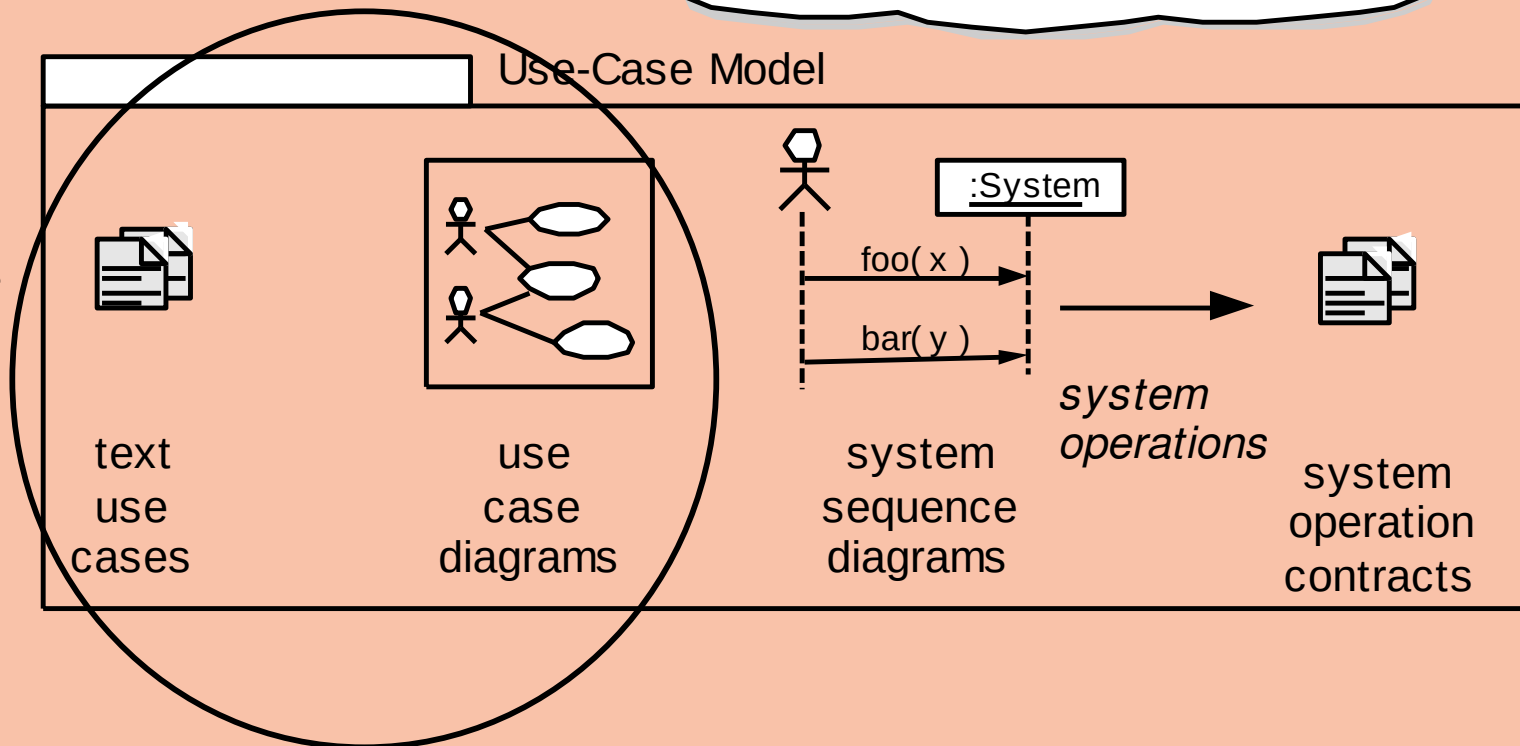
### 2a. Scanner failed.

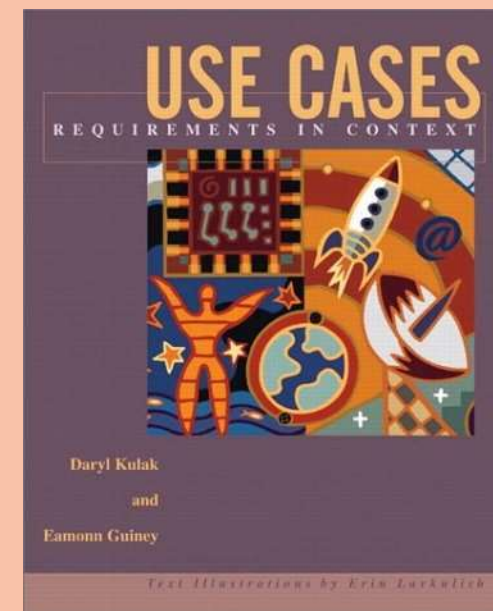
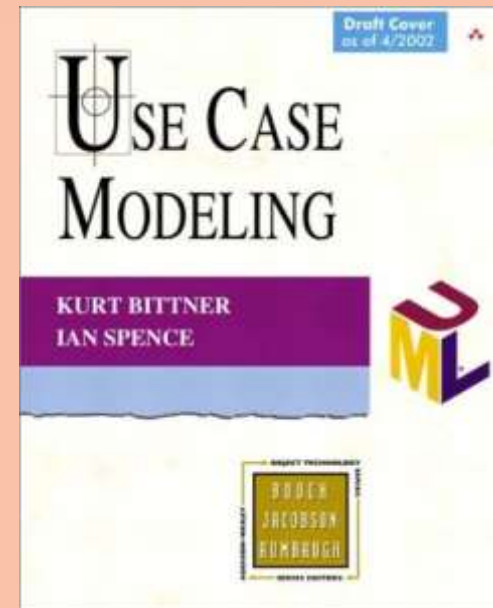
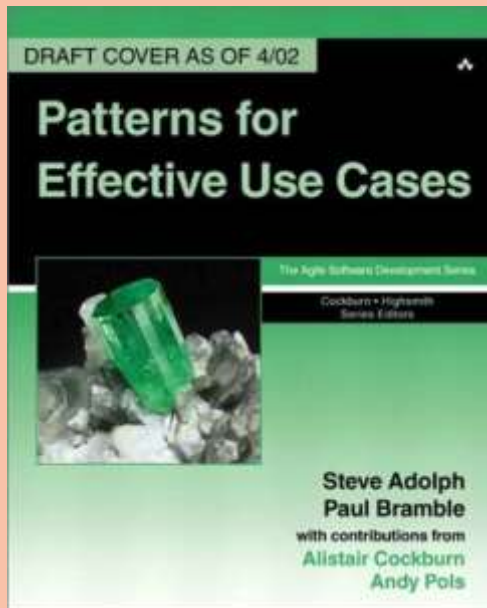
1. Clerk enters ID on keyboard (see GUI window example, fig 5)...



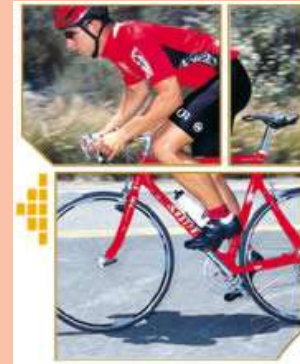
*Partial artifacts, refined in each iteration.*

**Requirements**

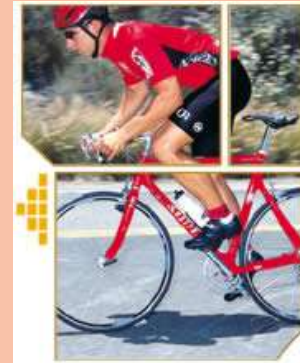




1. Objectives assessment. Can you:
  - Identify different use case levels
  - Write use cases in the popular Cockburn format
  - Contrast essential and concrete use cases
  - Apply use case guidelines
  - Read and write use case diagrams



1. Without notes, draw all UP disciplines and artifacts discussed.



1. Review the module and identify the guidelines for:
  - Use case modeling
  - Use case diagrams
  - Use case writing
2. Tell a partner, without looking at notes.
3. Reverse.

